

Auditoria Técnica Reversa

Real Estate Mining — Mapeamento Completo do Estado Atual

Gerado em: 03/06/2026 | Arquiteto de Sistemas / Engenharia Cognitiva

Resumo executivo: Plataforma de mineração de oportunidades imobiliárias em leilões (Brasil e EUA): scraping → persistência → extração de PDF → API REST → dashboard React. Maturidade estimada: **35–40%** em direção a produção.

1. Arquitetura Geral e Stack Tecnológica

Padrão arquitetural

Camada	Padrão
Backend	Monólito modular Django (apps: properties, scrapers, pdf_processor)
API	REST via Django REST Framework (ViewSet + Router)
Frontend	SPA React (Create React App)
Assíncrono	Workers Celery (filas scraping, pdf_processing)
Deploy	Docker Compose multi-serviço + Nginx reverse proxy

Não é microserviços: um codebase Django com processos separados (API, worker, beat). Separação pragmática por apps + Template Method em BaseScraper.

Stack — Backend (requirements.txt)

Pacote	Versão	Papel
Django	4.2.9	ORM, admin, WSGI
djangorestframework	3.14.0	API REST
django-filter	23.5	Filtros na API
django-cors-headers	4.3.1	CORS para React
celery	5.3.6	Tasks assíncronas
django-celery-beat	2.5.0	Agendamento (DB scheduler)
django-celery-results	2.5.1	Resultados no PostgreSQL
redis	5.0.1	Broker Celery
psycopg2-binary	2.9.9	PostgreSQL
gunicorn	21.2.0	Servidor WSGI
httpx	0.26.0	HTTP async nos scrapers
playwright	1.41.0	Scraping JS-heavy (EUA)

Pacote	Versão	Papel
beautifulsoup4	4.12.2	Parsing HTML (BR)
pdfplumber / pytesseract	—	PDF + OCR fallback

Stack — Frontend e Infra

Frontend: React 18, react-scripts 5, Tailwind 3.4. Infra: PostgreSQL 15, Redis 7, Nginx Alpine, WireGuard (linuxserver), Python 3.11-slim, Node 20-alpine. Config: .env / .env.example. Ausente: CI/CD, README raiz, pyproject.toml.

2. Mapeamento da Estrutura de Diretórios

```
real-estate-mining/  
■■■ backend/ # Monólito Django  
■ ■■■ config/ # settings, urls, celery, wsgi  
■ ■■■ apps/  
■ ■ ■■■ properties/ # Modelos, API, admin, filtros  
■ ■ ■■■ scrapers/ # Base scraper, fontes BR/US, tasks  
■ ■ ■■■ pdf_processor/ # Download + OCR + regex dívidas  
■ ■■■ media/pdfs/ # PDFs baixados  
■ ■■■ staticfiles/ # collectstatic (vendor)  
■ ■■■ Dockerfile  
■■■ frontend/ # Dashboard React + Tailwind  
■ ■■■ src/components/ # Dashboard, filtros, cards, stats  
■■■ nginx/ # Reverse proxy (:80)  
■■■ vpn/ # wg0.conf WireGuard  
■■■ docker-compose.yml
```

Pasta	Papel no ciclo de vida
backend/config	Bootstrap Django/Celery; rotas /api/ e /admin/
apps/properties	Domínio central: imóveis, fontes, API pública
apps/scrapers	Ingestão externa → upsert no banco
apps/pdf_processor	Enriquecimento pós-scrape via PDF
frontend	UI de descoberta, filtros e workflow de status
nginx	Gateway HTTP: API, admin, media, SPA
vpn	Túnel; celery_worker usa network_mode do WireGuard

3. Funcionalidades Implementadas

Módulo	Status	Descrição
Modelagem imóveis	OK	Property + enums país/leilão/dívida/status
Fontes scraping	OK	ScrapingSource + scraper_class dinâmico
Scraping BR	Template	BRLeiloeiroScraper — URL/seletores exemplo
Scraping US	Template	USCountyScraper — Playwright + anti-bot básico
Persistência	OK	update_or_create (source, external_id)
Pipeline PDF	OK	Download → pdfplumber → OCR → regex PT/EN
API REST	OK	CRUD + filtros + stats + trigger_pdf
Celery Beat	OK	06:00 → schedule_active_scrapers
Admin Django	OK	Gestão fontes e imóveis
Dashboard React	OK	Listagem, filtros, PATCH status
Autenticação	Ausente	API aberta (AllowAny)
Notificações	Ausente	—

Fluxo A — Scraping agendado

Celery Beat (06:00) → schedule_active_scrapers → run_scraper_task [VPN] → import dinâmico scraper_class → fetch_listings/detail → Property.update_or_create → (se pdf_url) process_pdf_task.delay

Fluxo B — Processamento PDF

process_pdf_task → download → media/pdfs/{uuid}.pdf → extract_text → key_values/debt_mentions → extra_data → preenche minimum_bid se vazio → pdf_processed=True

Fluxo C — Dashboard

Browser → Nginx :80 → /api/* → Gunicorn → PropertyViewSet → PostgreSQL → JSON → React fetch

Fluxo D — Workflow manual

PropertyCard → PATCH /api/properties/{id}/ {status} → PropertyUpdateSerializer → refresh lista/stats

4. Modelagem de Dados e Entidades

Relacionamento principal

ScrapingSource (1) — (N) **Property**. FK source com SET_NULL. Unique: (source, external_id). Índices: país/cidade, leilão/data, status/roi, moeda/lance.

ScrapingSource

- name, url, country, scraper_class (module.Class)
- is_active, last_scraped_at, schedule_cron (não usado pelo Beat atual)
- config JSON (ex.: county no scraper US)

Property — grupos de campos

Grupo	Campos principais
Proveniência	source, external_id, source_url, scraped_at
Localização	country, state, city, county, address, zip, lat/long
Imóvel	property_type, area_sqm/ft, bedrooms, bathrooms
Leilão	auction_type, auction_date, auction_number, process_number
Financeiro	*_cents (centavos), currency, debt_type, debt_details JSON
Análise	status, estimated_profit_pct (auto), roi_score (manual), notes
PDF	pdf_url, pdf_file, pdf_extracted_text, pdf_processed, extra_data

Cálculo ROI (implementado no save)

$$\text{estimated_profit_pct} = ((\text{market_value_cents} - (\text{minimum_bid_cents} + \text{total_debt_cents})) / \text{total_cost}) * 100.$$
 Requer market_value_cents preenchido — frequentemente vazio após scrape.

5. Infraestrutura, Deploy e Ecossistema

Serviço Docker	Função
postgres	BD principal + healthcheck
redis	Broker Celery
wireguard	VPN para scraping
api	Gunicorn :8000, 4 workers
celery_worker	network_mode: wireguard — scraping via VPN
celery_beat	Scheduler DB + crontab 06:00 fixo
frontend	CRA dev server :3000
nginx	Gateway :80 — /api, /admin, /media, /

Volumes

postgres_data, redis_data, pdf_storage

Backend Dockerfile

Poppler, Tesseract (por+eng), libs Chromium, playwright install chromium. Único settings: development (manage.py, wsgi, celery). Sem HTTPS no compose.

Ausências operacionais

- Pipeline CI/CD
- Testes automatizados
- Migrations além de 0001_initial
- Health endpoints / monitoring
- staticfiles/ versionado no repositório

6. Análise Crítica e Próximos Passos

Pontos fortes

- Domínio bem modelado (centavos, enums, índices, JSON flexível)
- Scraping extensível (BaseScraper + carregamento dinâmico)
- Filas Celery separadas com retries
- VPN dedicada ao worker de scraping
- Pipeline PDF dual (texto + OCR + regex bilíngue)
- Frontend funcional alinhado à API
- Docker Compose reproduzível

Gargalos e inconsistências

Área	Problema
Scrapers	URLs/seletores placeholder — sem dados reais
Segurança	API sem auth; DEBUG no .env.example
Produção	Só development settings; frontend via npm start
schedule_cron	Campo no modelo não usado pelo Beat
roi_score	Ordenação padrão mas nunca calculado automaticamente
pdf_file	FileField não populado pela task
Paginação	page_size=20 no front ignorado pelo DRF
Testes / Docs	Zero testes; sem README raiz; sem OpenAPI

Checklist — Crítico (bloqueante)

- Scrapers reais por fonte (seletores, rate limit, CAPTCHA)
- Autenticação API (JWT/session) + permissões DRF
- settings/production.py (DEBUG=False, security headers)
- Build estático React no Nginx (não CRA dev)
- HTTPS
- Testes + CI

Checklist — Alto impacto

- Regra clara para market_value_cents / roi_score
- schedule_cron por fonte ou PeriodicTask
- Associar PDF ao pdf_file
- Seeds ScrapingSource
- Remover staticfiles do git; collectstatic no deploy
- Backup PostgreSQL + PDFs

Checklist — Desejável

- Notificações (ROI > X)
- Geocodificação e mapa
- Página de detalhe no front
- OpenAPI + health /api/health/
- Monitoramento (Sentry, Prometheus)

Documento gerado automaticamente a partir da auditoria técnica reversa do repositório real-estate-mining. Conteúdo reflete o estado do código na data de geração.